Examen curs 08.06.2022

* Required

1. Email *

2. Nume si Prenume (First name & last name) *

3. Numar matricol (faculty ID) *

Probleme (Problems)

4. [UP|DE] Ce puteti spune despre executia urmatorului program? (What can you say about the execution of the next program? 1 point

```
1   #include <iostream>
2   class Data
3   {
4       protected: int x = 20;
5       public: int y;
6   };
7   int main()
8   {
9       Data d;
10      d.y = 10;
11      std::cout << d.x << ", " << d.y;
12      return 0;
13  }
```

Mark only one oval.

- ( ) Afiseaza 10, 20 / Will print 10, 20
- ( ) Afiseaza 20,10 / Will print 20, 10
- (✓) Nu compileaza pentru ca "Data::x" este inaccesibil / Will not compile because "Data::x" is inaccesible
- ( ) Afiseaza 10,10 / Will print 10,10
- ( ) Nu compileaza pentru ca nu exista un constructor care sa initializeze membru protected / Will not compile because Data class does not have a ctor that can initialized the protected members.
- ( ) Nu compileaza pentru ca d.y nu poate fi trimis pe un stream folosind operatoul << / Will not compile because d.y can not be send through a stream using << operator

5. [DE] Care dintre urmatoarele declaratii este o forma corecta de a defini un operator de indexare ? / Which one of the following statements is a valid index operator definition ? 1 point

Mark only one oval.

- ✓ int operator[] (double);
- int operator[] (int, int);
- int operator() (int)
- int operator() (int, int)
- int operator() ();

6. Ce se intampla la executia urmatorului program? What will happen when the next program is executed? 1 point

```
1  #include <iostream>
2  using namespace std;
3
4  class Masina {
5  protected:
6      int km;
7  public:
8      void SetKM() { km = 10; };
9      int GetKM() { return km; };
10 };
11
12 class Dacia : public Masina {
13 protected:
14     int km;
15 public:
16     void SetKM() { km = 20; };
17 };
18
19 int main() {
20     Dacia d;
21     Masina* m = (Masina*)&d;
22     m->SetKM();
23     d.SetKM();
24     cout << m->GetKM() << " " << d.GetKM();
25     return 0;
26 }
```

Mark only one oval.

- ( ) programul compileaza, dar da eroare la executie (the code is compiled, however a runtime exception is triggered)
- ( ) programul nu compileaza, "redefinire km" (the code does not compile, " km redefinition")
- (✓) afiseaza pe ecran 10 10 (10 10 is printed)
- ( ) afiseaza pe ecran 20 20 (20 20 is printed)
- ( ) afiseaza pe ecran 10 20 (10 20 is printed)
- ( ) afiseaza pe ecran 20 10 (20 10 is printed)

***al doilea 10 pentru ca este apelata metoda GetKM din clasa parinte, deci acceseaza variabila km care apartine clasei parinte, nu km din clasa derivata**
Cand este executata linia 22, ea se executa pe portiunea din clasa base a obiectului derivat, deci se seteaza km cu 10, iar cand se apeleaza d.GetKM() e afisat tot 10 pentru ca metoda apartine clasei parinte si acceseaza km din partea clasei parinte din interiorul obiectului (din memorie) derivat, nu km din clasa derivata; de mentionat ca clasa derivata nu face overwrite lui km din clasa parinte

7. [DE] Ce afiseaza executia urmatorului program? / What is the output when the following program is executed? 1 point

```
#include <exception>
#include <iostream>
using namespace std;

class Division : public exception
{
    const char* what() const throw() {
        return "Division by 0";
    }
};

int main() {
    int imp = 0;
    cout << "Inainte de exceptie" << endl;
    try {
        cout << "7 / 0 = " << 7 / imp << endl;
    }
    catch (exception& e) {
        cout << e.what();
    }
    cout << "Dupa exceptie" << endl;
    return 0;
}
```

Mark only one oval.

- ( ) Nu afiseaza nimic / It displays nothing
- ( ) Afiseaza doar "Division by 0" / It only displays "Division by 0"
- (✓) Afiseaza doar "Inainte de exceptie" / It only displays "Inainte de exceptie"
- ( ) Afiseaza doar "Dupa exceptie" / It only displays "Dupa exceptie"
- ( ) Afiseaza toate mesajele / It displays all possible messages

***da crash cu 0 division pentru ca catch prinde numai erorile trimise prin throw!**

8. [UP] Cum se pot crea instante la clasa TEST? / How to create instances of the TEST class? 1 point

```
3   class TEST {
4   private:
5       TEST();
6       ~TEST();
7       static TEST* test;
8   public:
9       static TEST* Metoda() {
10          if (!test)
11              test = new TEST();
12          return test;
13      }
14  };
```

Mark only one oval.

- ( ) TEST inst;
- ( ) TEST* inst = new TEST;
- ( ) TEST inst = TEST.Metoda();
- ( ) TEST* inst = TEST->Metoda();
- ( ) Nu putem face instante, deoarece constructorul si destructorul sunt privati / We cannot create instances, as the constructor and destructor are declared private
- (✓) TEST* inst = TEST::Metoda();

***conceptual, noi am avut probleme punand prostia asta in practica; ne-am certat cu proful si a zis ca nu conteaza ca nu compileaza, conceptual/sintactic e corect, bai lasa-ma**

9. Ce afiseaza urmatorul program? (What does the program display?) 1 point

```
int main()
{
    std::vector<int> elements{ 1, 3, 5, 6, 3 };
    int sum = 0;
    for (auto it = elements.begin(); it < elements.end(); it++)
    {
        if (it != elements.begin())
            sum += (*it - 1) + *(it - 1);
    }
    std::cout << sum;
    return 0;  ≤ 1ms elapsed
}
```

Mark only one oval.

- ( ) 26
- (✓) 28
- ( ) 15
- ( ) 30
- ( ) 19

10. [DE] Care dintre urmatoarele afirmatii referitoare la executia urmatorului program este adevarata ? / Witch one of the following statements about the execution of the next program is true ? 1 point

```
1   #include <iostream>
2   using namespace std;
3
4   class B {
5       int x = 0;
6   public:
7       B() { cout << "B::B()" << endl; }
8       ~B() { cout << "B::~B()" << endl; }
9       virtual void C() { x = x * 10; }
10      virtual int G() { return x; }
11  };
12
13  class D : public B {
14      int y = 0;
15  public:
16      D() { cout << "D::D()" << endl; }
17      ~D() { cout << "D::~D()" << endl; }
18      void C() { y = y + 50; }
19      int G() { return y; }
20  };
21
22  int main()
23  {
24      B* b = new D();
25      b->C();
26      cout << b->G() << endl;
27      delete b;
28      return 0;
29  }
```

Mark only one oval.

- [ ] Se va executa corect succesiunea de constructori/destructori atat pentru B cat si pentru D / The sequence of constructors and destructors will be successfully called for both B and D class
- [ ] Va lipsi apelul constructorului pentru B / There will be missing the call for B constructor
- [ ] Va lipsi apelul constructorului pentru D / There will be missing the call for D constructor
- [ ] Va lipsi apelul destructorului pentru B / There will be missing the call for B destructor
- [x] Va lipsi apelul destructorului pentru D / There will be missing the call for D destructor

***trebuie facut virtual destructorul din B ca sa apara si cel din D pentru ca accesand un obiect derivat printr-un pointer base, compilatorul va avea acces numai la destructorul din base**

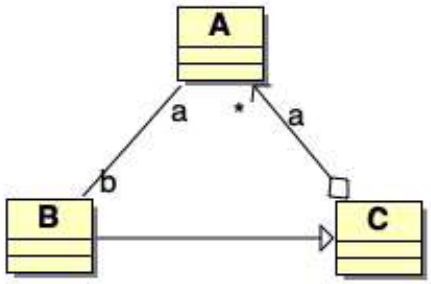
11. Cum se va comporta urmatorul program? / How will this application behave? 1 point

```cpp
#include <iostream>
template <typename Fn>
struct Generator {
    Fn fn;
    Generator(Fn fn) : fn(fn) {
    }
    unsigned gen() {
        return fn();
    }
};
int main() {
    auto fn  = [count = 5]() mutable { return count--; };
    auto gen = Generator<decltype(fn)>(fn);
    for (auto i = gen.gen(); i != 0; i = gen.gen()) {
        std::cout << i << ' ';
    }
}
```

Mark only one oval.

- ( ) afiseaza 5 4 3 2 1 0 / prints 5 4 3 2 1 0
- (✓) afiseaza 5 4 3 2 1 / prints 5 4 3 2 1
- ( ) eroare de compilare: template / compiler error: template
- ( ) eroare de compilare: lambda / compiler error: lambda
- ( ) eroare de compilare: for / compiler error: for

12. [UP] Care afirmații sunt adevărate despre următoarea diagramă de clase? What claims are true about the following class diagrams? 1 point

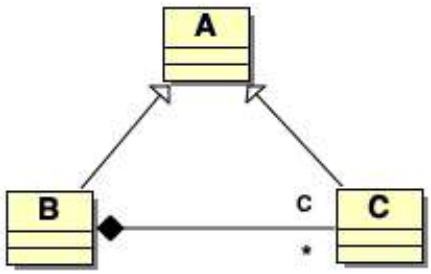


Mark only one oval.

- ( ) Există o relație de asociere unidirecțională între C și A. / There is a unidirectional association relation between C and A.
- (✓) Există o relație de asociere bidirecțională între B și A. / There is a bidirectional association relation between B and A.
- ( ) Există o relație de agregare între B și C. / There is an aggregation relation between B and C.
- (✓) Există o relație de agregare între C și A. / There is an aggregation relation between C and A.
- (✓) Există o relație de generalizare/specializare între C și B. / There is a generalization/specialization relation between C and B.
- ( ) Există o relație de generalizare/specializare între C și A. / There is a generalization/specialization relation between C and A.

Oricare dintre cele trei raspunsuri daca este selectat va da punctul complet la aceasta intrebare.

13. [UP] Ce principiu nu e respectat în proiectarea claselor din diagramă? / What principles are violated in the next class diagram? 1 point



Mark only one oval.

- ( ) Principiul închis/deschis, ierarhia nu se poate extinde. / Open-close principle, the hierarchy cannot be extended.
- ( ) Principiul lui Liskov, deoarece B și C nu sunt subtipuri comportamentale ale lui A. / The Liskov Substitution Principle, because B and C are not behavioral subtypes of A.
- ( ) Principiul unei singure responsabilități, deoarece B si C deriveaza din A.
- (✓) Principiul inversării dependențelor, deoarece există o agregare între B și C, care induce o dependență.

This content is neither created nor endorsed by Google.

Google Forms